

# Google Gemini API Integration with Python

- ▶ **Building AI-Powered Applications using Google Generative AI**

## **Tools Used:**

- Python
- Jupyter Notebook
- Google Gemini API

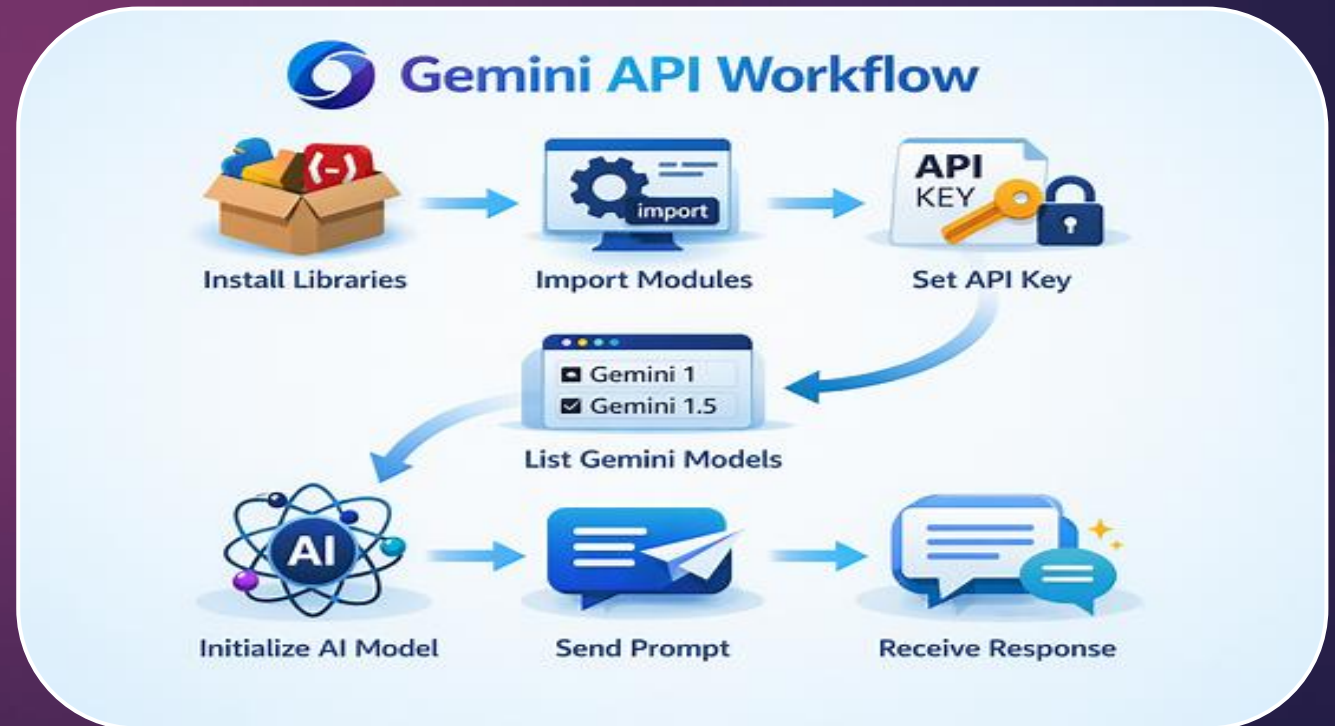
Name: Harpreet Kaur  
Department: B tech CSE  
College: JMIETI

# Project Overview

This project demonstrates how to integrate Google's Gemini API into a Python application using the official Google Generative AI library.

The notebook covers:

- Installing the required libraries
- Importing necessary modules
- Configuring the Gemini API key
- Listing available Gemini models
- Initializing a Generative AI model
- Sending prompts to the model
- Receiving AI-generated responses



# Step 1: Installing Required Libraries

## Objective:

Before interacting with the Google Gemini API, the necessary Python libraries must be installed.

## Libraries Used:

```
graph TD; A[Libraries Used:] --> B[Google-GenerativeAI:]; A --> C[Openai-whisper:];
```

### Google-GenerativeAI:

- Connect to Gemini models
- Generate AI-powered responses

### Openai-whisper:

- Voice assistants
- Audio transcription
- Speech-to-text applications

```
[2]: %pip install -q google-generativeai openai-whisper
```

Required packages are installed successfully before using the Gemini API.

# Step 2: Importing the Google Generative AI Library

## **import google.generativeai as genai:**

Imports the **Google Generative AI SDK** into the notebook.

The alias **genai** is used to simplify future API calls. This library allows developers to configure the API, access Gemini models, and generate AI-powered responses.

```
[8]: import google.generativeai as genai  
     print(genai.__version__)
```

0.8.6

## **print(genai.\_\_version\_\_):**

- Displays the installed version of the **Google Generative AI** package.
- Helps verify that the library has been installed correctly.
- Ensures compatibility with the notebook before proceeding with API integration.

# Step 3: Configuring the API Key and Exploring Gemini Models

```
[9]: genai.configure(api_key="AQ.Ab8RN6LE604qMJP2jMMenPRnLgyEWhkBOW2U0dmbcuJN0TugRg")

for model in genai.list_models():
    if "generateContent" in model.supported_generation_methods:
        print(model.name)
```

## Sample Output:

```
models/gemini-2.5-flash
models/gemini-2.5-pro
models/gemini-2.0-flash
models/gemini-2.0-flash-001
models/gemini-2.0-flash-lite-001
models/gemini-2.0-flash-lite
models/gemini-2.5-flash-preview-tts
models/gemini-2.5-pro-preview-tts
models/gemma-4-26b-a4b-it
models/gemma-4-31b-it
models/gemini-flash-latest
models/gemini-flash-lite-latest
models/gemini-pro-latest
models/gemini-2.5-flash-lite
```

**genai.configure(api\_key=...)** authenticates the application using the Gemini API key.

**genai.list\_models()** retrieves all available Gemini models. The if condition filters models that support content generation. **print(model.name)** displays the names of the supported models.

Successfully connects to the Gemini API.

# Step 4 – Loading the API Key Securely

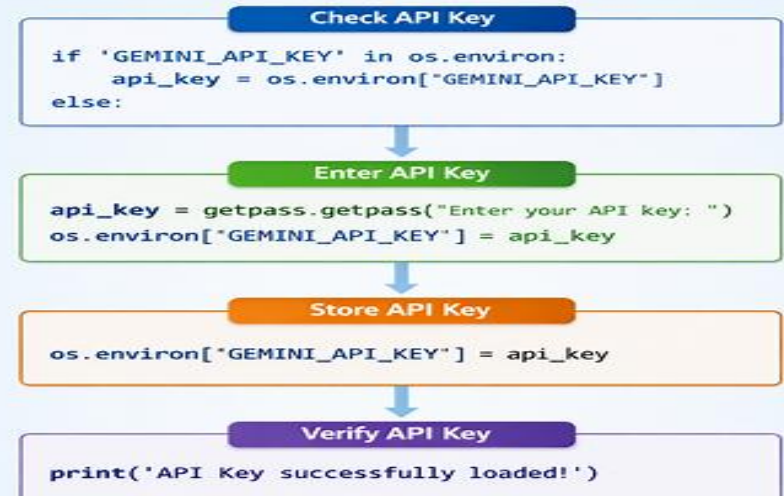
```
[13]: import os
import getpass

if not os.environ.get("GEMINI_API_KEY"):
    os.environ["GEMINI_API_KEY"] = getpass.getpass("AQ.Ab8RN6LE6O4qMJP2jMMenPRnLgyEWhkBOW2UOdmbcuJN0TugRg")
print("API_KEY_LOADED:", "GEMINI_API_KEY" in os.environ)

API_KEY_LOADED: True
```

- `os.environ` checks whether the **GEMINI\_API\_KEY** already exists.
- `getpass.getpass()` securely accepts the API key without displaying it on the screen.
- The API key is stored as an **environment variable**.
- The `print()` statement verifies that the API key has been successfully loaded.

## Configuring the Gemini API Key





# Step 5: Initializing the Gemini Model and Generating a Response

```
[22]: # configure the api key
genai.configure(api_key=os.environ["GEMINI_API_KEY"])
model = genai.GenerativeModel("gemini-2.5-flash")
response=model.generate_content(
    "Tell me what is an API? What is its use?"
)
print(response.text)
```

## Output:

An **API** (Application Programming Interface) is a set of rules, protocols, and tools that allows different software applications to communicate with each other.

### What is its use?

APIs are the backbone of modern software development and are used everywhere you look. Here are its primary uses:

- Enabling Communication & Integration:**
  - Connecting different services:** This is the most fundamental use. APIs allow disparate systems to talk to each other.
  - Example:** When you buy something online, the e-commerce website uses a payment gateway API (like Stripe or PayPal) to process your credit card without having to build its own secure payment system.
  - Example:** Embedding a Google Map on a website. Your website uses Google Maps' API to display the map data.

- **genai.configure()** uses the API key stored in the environment variable to authenticate the application.
- **GenerativeModel("gemini-2.5-flash")** initializes the Gemini 2.5 Flash model.
- **generate\_content()** sends the user prompt to the model.
- **print(response.text)** displays the AI-generated response in the notebook.

# Conclusion

This project successfully demonstrates the integration of the **Google Gemini API** with Python to generate AI-powered responses. The workflow included installing the required libraries, configuring the API key securely, exploring available Gemini models, initializing the **Gemini 2.5 Flash** model, and generating responses from user prompts.

- Successfully integrated the Google Gemini API with Python.
- Used environment variables for secure API key management.
- Explored and selected an appropriate Gemini model.
- Generated accurate and meaningful AI responses using prompts.



Thank  
you!

